

운영체제 실습

Assignment #5

Class : A
Professor : 김태석 교수님
Student ID : 2020202090
Name : 최민석

Introduction

Assignment5-1에서는 /proc 디렉토리에 하위 디렉토리 proc_2020202090 을 생성하고 거기에 프로세스 정보를 표시하는 processInfo 파일을 생성하여 전체 프로세스 혹은 원하는 프로세스의 pid, ppid, uid, gid, utime, stime, state 를 파일에 표시하는 모듈을 구현한다. 이 모듈은 적재 시 /proc/proc_2020202090/processInfo 를 생성하고 모든 프로세스의 정보를 pid=1 부터 오름차순으로 기록한다. 모듈이 해제될 때 생성한 디렉토리 와 processInfo 파일을 삭제한다.

Assignment5-2에서는 FAT (file allocation table)을 기반으로 한 가상 파일 시스템을 구현해본다. 파일 생성, 데이터 쓰기, 읽기, 파일 삭제 및 파일 리스트 출력을 수행할 수 있어야 하며 메모리에서 상태를 관리하고 파일에서 상태를 저장 및 복원할 수 있어야 한다. FAT 은 파일을 디스크에 저장하는 방식을 테이블로 매핑하는 파일 시스템 구조로, 각 파일을 블록으로 나누고 FAT 테이블 상에서 서로 연결되게 한다.

구현해야 할 myfat 시스템은 FAT 테이블, 파일 엔트리, 데이터 블록 배열을 포함해야 한다. FAT 테이블은 파일에 속한 데이터 블록을 추적하는 매핑 테이블로, 테이블의 각 항목은 데이터 블록에 해당하며 파일의 다음 블록을 가리킨다. 파일의 마지막 블록은 -1 로 표시한다. 파일 엔트리는 파일 이름, 크기, 첫번째 블록 위치 등의 메타 데이터를 포함한다. 파일의 실제 데이터는 데이터 블록 배열에 기록한다. 각 블록은 고정된 512 바이트의 크기를 가지며 파일이 블록 크기를 초과하는 경우 추가 블록을 할당하며 FAT 테이블을 통해 연결한다.

파일 시스템을 FAT 기반으로 구현하며 메모리에서 파일을 관리하고 프로그램 종료 시 파일 시스템 상태를 디스크에 저장한다. 과제 pdf 파일에 제공된 함수 정의에 따라 파일 생성, 읽기, 쓰기, 삭제 기능을 구현한다. 파일 시스템의 상태가 디스크에 저장되고 프로그램 재시작 시 복원될 수 있도록 한다.

최대 파일 개수는 100 개, 데이터 블록의 개수는 1024 개, 블록 사이즈는 32 바이트, 최대 파일 이름 길이는 100 자로 제한한다.

결과화면

- Assignment 5-1

모듈을 적재한 직후에는 processInfo 파일에 아무것도 적지 않은 상태이므로 모든 프로세스의 정보가 기록된다.

```
os2020202090@ubuntu:~/test1$ sudo insmod proc_info.ko
os2020202090@ubuntu:~/test1$ cat /proc/proc_2020202090/processInfo
PID      PPID      UID        GID        UTIME      STIME      STATE      NAME
-----
1         0         0          0           804000000  3044000000 S (sleeping) systemd
2         0         0          0           0          160000000 S (sleeping) kthreadd
3         2         0          0           0           0          I (idle) rcu_gp
4         2         0          0           0           0          I (idle) rcu_par_gp
6         2         0          0           0           0          I (idle) kworker/0:0H
8         2         0          0           0           0          I (idle) mm_percpu_wq
9         2         0          0           0          360000000 S (sleeping) ksoftirqd/0
10        2         0          0           0          640000000 I (idle) rcu_sched
11        2         0          0           0          960000000 S (sleeping) migration/0
12        2         0          0           0           0          S (sleeping) idle_inject/0
14        2         0          0           0           0          S (sleeping) cpuhp/0
15        2         0          0           0           0          S (sleeping) cpuhp/1
16        2         0          0           0           0          S (sleeping) idle_inject/1
17        2         0          0           0          840000000 S (sleeping) migration/1
18        2         0          0           0          520000000 S (sleeping) ksoftirqd/1
```

1 번 프로세스부터 오름차순으로 모든 프로세스의 pid, ppid, uid, gid, utime, stime, state, name 정보가 표시된다.

```
os2020202090@ubuntu:~/test1$ echo 1 > /proc/proc_2020202090/processInfo
os2020202090@ubuntu:~/test1$ cat /proc/proc_2020202090/processInfo
PID      PPID      UID        GID        UTIME      STIME      STATE      NAME
-----
1         0         0          0           804000000  3052000000 S (sleeping) systemd
os2020202090@ubuntu:~/test1$
```

cat 의 리다이렉션으로 열람할 pid=1 을 processInfo 파일에 입력하고 다시 processInfo 파일을 확인하면 해당 프로세스의 정보만 표시된 것을 볼 수 있다.

```
os2020202090@ubuntu:~/test1$ sudo rmmod proc_info.ko
os2020202090@ubuntu:~/test1$ cat /proc/proc_2020202090/processInfo
cat: /proc/proc_2020202090/processInfo: No such file or directory
os2020202090@ubuntu:~/test1$
```

모듈을 적재 해제한 후 processInfo 파일을 확인하려 하면 모듈 exit 함수에서 클린업을 진행했기 때문에 디렉토리와 파일이 사라진 것을 볼 수 있다.

- Assignment 5-2

파일 생성, 파일 목록 열람, 파일에 작성, 파일 읽기, 파일 삭제 등의 기능을 구현한 FAT 기반 가상 파일 시스템이다. 예제는 과제 pdf 파일을 참조하였다.

```
os2020202090@ubuntu: ~/test2
os2020202090@ubuntu:~/test2$ ./fat create A
Error: No saved state found. Starting fresh.
File 'A' created.
os2020202090@ubuntu:~/test2$ ./fat create B
File 'B' created.
os2020202090@ubuntu:~/test2$ ./fat list
Files in the file system:
File: A, Size: 0 bytes
File: B, Size: 0 bytes
os2020202090@ubuntu:~/test2$ ./fat write A "Hello, world"
Data written to 'A'.
os2020202090@ubuntu:~/test2$ ./fat list
Files in the file system:
File: A, Size: 12 bytes
File: B, Size: 0 bytes
os2020202090@ubuntu:~/test2$ ./fat write B "Helo, world"
Data written to 'B'.
os2020202090@ubuntu:~/test2$ ./fat write B "Hola, world!"
Data written to 'B'.
os2020202090@ubuntu:~/test2$ ./fat list
Files in the file system:
File: A, Size: 12 bytes
File: B, Size: 23 bytes
os2020202090@ubuntu:~/test2$ ./fat read A
Content of 'A': Hello, world
os2020202090@ubuntu:~/test2$ ./fat read B
Content of 'B': Hola, world!
os2020202090@ubuntu:~/test2$ ./fat delete B
File 'B' deleted.
os2020202090@ubuntu:~/test2$ ./fat list
Files in the file system:
File: A, Size: 12 bytes
os2020202090@ubuntu:~/test2$
```

파일 시스템이 정상적으로 동작하며 주어진 예시 외에도 너무 큰 데이터를 쓰려고 시도하거나 유효하지 않은 명령어 입력에 대해서도 테스트하였다.

고찰

- Assignment 5-1

처음에는 기존에 수행했던 모듈 프로그래밍들처럼 336 번 시스템 콜을 후킹하여 기능을 구현하는 줄 알았으나, cat 명령어와 리다이렉션을 통해 /proc/proc_2020202090/processInfo 파일에 입출력을 수행하도록 구현하는 것을 깨달았다.

모듈이 적재된 초기에는 processInfo 파일이 생성된 직후라 내용이 없으므로 자동적으로 모든 프로세스의 정보를 기록하게 된다.

cat /proc/proc_2020202090/processInfo 로 이 정보를 확인한 후 echo (pid) > /proc/proc_2020202090/processInfo 를 통해 해당 파일에 열람할 프로세스의 pid 값을 입력하면 새롭게 프로세스 정보를 파일에 작성한다.

모듈이 적재된 동안 반복적으로 특정 프로세스의 정보를 표시할 수 있기 때문에, 이전에 커널에 정보를 직접 출력하도록 구현했던 방식보다 더 효율적이라고 생각했다. 이전 방식은 커널을 읽는 범위를 잘못 설정하면 유효하지 않은 정보를 출력할 가능성이 있었지만, 지금은 항상 파일에 유효한 정보를 출력하기 때문이었다.

- Assignment 5-2

처음에는 이전에 구현했던 FTP 시스템처럼 파일을 직접 생성하고 관리하는 줄 알았지만, 가상 파일 시스템이라는 점에 주목하였더니 assignment 의 출제 의도를 이해할 수 있었다.

가상머신의 경우 직접 운영체제를 컴퓨터에 설치하지 않고도 프로그램의 메모리와 인스턴스, 상태 정보만을 통해 시스템을 제공하므로, 가상 파일 시스템도 실제로 파일을 저장하는 것이 아닌, 메모리와 상태 정보를 저장하고 복원하여 운용하는 것이라는 것을 알게 되었다.

따라서 파일을 직접 디렉토리에 저장하지 않고 메모리 상에 가상으로 유지시키면서 이 가상 파일 시스템의 정보만 전부 저장 및 복원하는 방식으로 구현하였다.

Reference

- 24-2_OSLab_Assignment-5_v1.pdf
- 24-2_OSLab_12_File_System-procfs.pdf
- 24-2_OSLab_12_Memory_management_v2.pdf
- 24-2_OSLab_13_Fat-based_VFS_Implementation.pdf